

 UTP Universidad Tecnológica del Perú	Curso: Algoritmos y Estructuras de Datos	Consigna del Proyecto Final
Docente: Sardón Paniagua, Aníbal (C16290@utp.edu.pe)		

Objetivo

Desarrollar una aplicación utilizando el lenguaje de programación Java y el paradigma de orientación a objetos utilizando las Estructuras de Datos y Algoritmos estudiados en el curso que solucione un problema presentado en un contexto.

Consideraciones generales

1. Cada equipo estará conformado por hasta tres (3) integrantes.
2. Cada equipo deberá designar un líder.
3. El trabajo final será entregado por los líderes de equipo en la plataforma UTP+Class.
4. Se entregará el código fuente del proyecto en un archivo ZIP.

Consideraciones técnicas

1. El proyecto se desarrollará utilizando un IDE con Java Swing, Maven y Java JDK 17, como mínimo.
2. El tipo de aplicación a desarrollar es una aplicación con **formularios Windows de Java Swing**.
3. Deberán aplicarse en el proyecto los temas tratados en el curso: **programación orientada a objetos, estructuras de datos dinámicas, Listas enlazadas circulares dobles, colas, pilas, árboles ABB, árboles AVL, para mostrar y filtrar la información que el usuario solicita.**
4. El programa utiliza una Base de Datos: **XML/JSON**
(No se permite otro origen de datos ni otro tipo de base de datos)
5. Los formularios de mantenimiento utilizarán las Librerías de Java para poder acceder al origen de datos (*archivos XML*) y realizar las operaciones de básicas.

6. Los formularios de consultas utilizarán como estructuras para almacenar datos temporales en memoria usando las estructuras de datos creadas.
7. No se puede utilizar la biblioteca de clases de Colecciones de Java como ArrayList, List, Map, Set y sus clases derivadas.
8. La información de cada elemento buscado estará encapsulada en objetos, es decir las estructuras de datos contendrán objetos.
9. Todas las estructuras de datos construidas estarán implementadas con Tipos Abstractos de Datos (TAD).
10. El código fuente deberá estar debidamente comentado.
11. Deberá incluirse en los comentarios de cada pieza de software el autor.
12. El programa estará **estructurado en Paquetes**.

Entregables

- Informe de trabajo final.
- Proyecto de Java (zip)
- Diapositivas de presentación (.ppt, .pptx)

Presentación

1. La sustentación tendrá lugar en la Semana 18.
2. Parte 1. Exposición: grabarán un vídeo que tendrá una duración de 15 minutos aprox. donde expondrán sobre la implementación de las estructuras de datos.
3. Parte 2. Preguntas del docente: duración variable. El docente preguntará sobre el código fuente y la lógica del programa.

Estructura de informe

- I. Carátula
(debe mostrar la facultad, carrera, curso, título del trabajo, ciclo, integrantes del equipo, docente, sede, año y el logo de UTP el inicio).
- II. Índice.
- III. **Capítulo 1.** Aspectos Generales.
 - a. Descripción del problema.
 - b. Objetivos del sistema.

IV. **Capítulo 2.** Diseño de la Aplicación.

a. Descripción de los formularios.

Nombre del formulario	Descripción	Usuarios
Mantenimiento de usuarios	Permite realizar las operaciones básicas de: - Consultas por usuario. - Registrar nuevos usuarios. - Modificar los datos de un usuario. - Eliminar usuarios.	- Administrador del sistema - Vendedor

[Realice la explicación de los principales eventos que se ejecutarán en el formulario]

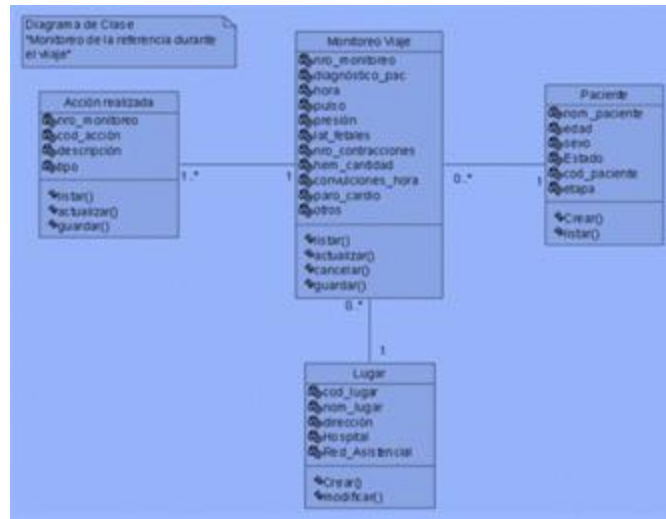
Descripción de los Eventos

Control	Título	Evento	Descripción
btnRegistrar	Registrar Cliente	Clic	Graba en el archivo “Cliente” un nuevo registro al final
btnModificar	Modificar Cliente	Clic	Reemplaza en el archivo “Cliente” los cambios realizados en un cliente.
BtnEliminar	Eliminar Cliente	Clic	Borra una fila en el archivo “cliente” correspondiente al cliente seleccionado.

b. Diseño de las clases.

(diagrama de clases y descripción de las funcionalidades de sus métodos)

Ejemplo:



Esquema:

Paquete: <nombre del paquete>

Clase: <nombre de la clase>

Atributos:

Atributo	Descripción

Métodos:

Método	Descripción

c. Diseño de las Estructuras de Datos.

Descripción de las estructuras de datos TADs usados utilizados (listas enlazadas, pilas, colas, árboles, etc) , se puede representar mediante gráficos.

- V. Conclusiones.
- VI. Recomendaciones.
- VII. Bibliografía.

Proyecto: Sistema de Control y Experiencia del Visitante "UTP Park Manager"

La corporación internacional "UTP Park" está diseñando su próximo parque temático hipertecnológico. Para ofrecer una experiencia premium, cada visitante recibe un brazalete electrónico inteligente que rastrea sus accesos, compras y reservas. El parque necesita un software centralizado que corra en servidores locales para procesar en tiempo real las colas de las atracciones, el mapa del parque, el sistema de créditos (puntos) de los usuarios y el control de mantenimiento de los juegos mecánicos.

Debido a que el sistema debe operar de forma ultra rápida ante cortes de energía o fluctuaciones de red, todo se gestionará en la memoria RAM del servidor usando estructuras de datos personalizadas en Java, con un módulo de respaldo continuo que serializa el estado del parque en archivos XML.

Requisitos Funcionales (RF)

- **RF-1: Registro y Control de Visitantes:** Registrar a los visitantes con un código de brazalete único, nombre, saldo disponible y tipo de pase (*Regular, VIP, FastPass*).
- **RF-2: Fila Virtual y Acceso a Atracciones:** Gestionar el ingreso a las atracciones. Los usuarios con pase VIP o FastPass deben tener prioridad de acceso sobre los pases regulares.
- **RF-3: Mapa Interactivos de Sectores (Zonas):** Permitir a los administradores organizar el parque por zonas (*Aventura, Fantasía, Sci-Fi*) y sub-zonas de forma jerárquica.
- **RF-4: Ciclo de Operación de Atracciones (Simulación):** Simular el funcionamiento de un juego mecánico (*ejemplo. una montaña rusa*), donde los carritos numerados circulan e ingresan a la estación de abordaje uno tras otro en un bucle infinito.
- **RF-5: Historial de Transacciones Financieras:** Registrar las compras o recargas de saldo de cada visitante, permitiendo anular la última transacción realizada (*por ejemplo, por un cobro duplicado o error de digitación*).
- **RF-6: Búsqueda y Validación de Brazaletes:** Validar de forma inmediata si un brazalete es válido y consultar los datos del visitante cuando este pasa por un sensor.
- **RF-7: Persistencia Integral en XML:** Guardar toda la configuración del parque, el árbol de atracciones y los datos de los usuarios en archivos .xml al cerrar la aplicación, y restaurarlos limpiamente al iniciar.

Módulos

1. Formulario Principal

- El formulario principal posee un menú principal desde la cual se puede acceder a los demás formularios del sistema de información.

2. Formulario Taquilla y Accesos:

- Controla el ingreso de visitantes, la creación de sus perfiles de brazalete y la carga de saldos.

3. Formulario para Atracciones y Operaciones:

- Gestiona las filas de espera físicas y virtuales de cada juego mecánico, además del ciclo técnico de los vagones/carritos.

4. Formulario para de Infraestructura (Mapa):

- Organiza la distribución física y temática del parque (Zonas, Restaurantes, Tiendas, Baños).

5. Formulario de Financiero y Reversiones:

- Monitorea los gastos que hacen los usuarios dentro del parque y gestiona la corrección de errores de facturación.

6. Persistencia de Datos:

- Todos los datos deben guardarse y cargarse utilizando archivos XML.
- El programa debe importar los datos desde los archivos XML, cargar los datos hacia las estructuras de datos y realizar las operaciones de búsquedas, ordenamiento, filtrado usando las estructuras de datos creadas.
- Dado que el estado completo del parque debe salvarse, deben diseñar una estrategia para serializar árboles y listas.

Especificaciones Técnicas del Programa

1. Árbol AVL (Módulo de Taquilla y Accesos)

- **Dónde se implementa:** En el índice central de Visitantes Activos en el parque.
- **Cómo se usa:** Cada nodo almacena la información del visitante (Brazalete, Nombre, Saldo). La clave de ordenamiento y balanceo es el código único del brazalete.
- Un parque puede albergar a 20,000 personas simultáneamente. Cuando un usuario pasa por un sensor, el sistema no puede tardar segundos buscando en una lista lineal. El Árbol AVL asegura que la validación ocurra de forma instantánea en tiempo logarítmico ($O(\log n)$).

2. Lista Enlazada Doble (Módulo de Atracciones - Fila Virtual)

- **Dónde se implementa:** En la lista de espera de reservas o turnos de una atracción en particular.
- **Cómo se usa:** Almacena los ID de los brazaletes que reservaron su turno para ese juego. Permite a los operadores mover a un usuario de posición en la fila si viene con una persona con movilidad reducida o removerlo si decide salirse de la fila de manera voluntaria antes de tiempo.
- Los punteros anterior y siguiente facilitan la eliminación o reordenamiento de nodos en medio de la lista en tiempo constante $O(1)$ una vez localizado el nodo, sin necesidad de desplazar elementos como en un arreglo.

3. Lista Enlazada Circular Simple (Módulo de Operaciones - Ciclo de Vagones)

- **Dónde se implementa:** En el sistema de control de los carritos de atracciones mecánicas (ej. Montaña Rusa o Carrusel).
- **Cómo se usa:** Cada nodo representa un vagón físico con capacidad limitada. El último vagón apunta al primero. El programa simula el despacho: el Vagón 1 sale a la pista, el Vagón 2 pasa a la estación de abordaje, y cuando el Vagón 1 regresa del recorrido, se posiciona automáticamente detrás del último vagón en espera.
- Modela perfectamente los sistemas físicos de ciclo cerrado y continuo, donde no hay un "final" real, sino un flujo rotativo perpetuo de recursos.

4. Cola de Prioridad (Módulo de Atracciones - Fila de Acceso Físico)

- **Dónde se implementa:** En los torniquetes de entrada de cada juego.
- **Cómo se usa:** Almacena a las personas que están haciendo la fila en ese preciso momento. El criterio de prioridad se define por el tipo de pase: VIP = Prioridad 3, FastPass = Prioridad 2, Regular = Prioridad 1.
- El sistema despachará siempre primero a los usuarios con mayor prioridad (VIP). Si dos usuarios tienen la misma prioridad, se respeta estrictamente el orden de llegada (comportamiento FIFO).

5. Pila (Módulo Financiero - Reversiones / Undo)

- **Dónde se implementa:** En la billetera virtual de cada visitante o en el módulo general de cajas.
- **Cómo se usa:** Cada vez que el visitante consume en un restaurante o tienda del parque, la transacción se apila (Push). Si el cajero se equivoca o el cliente cancela la compra inmediatamente, el sistema ejecuta un Pop para extraer la última transacción (LIFO) y devolver el dinero al saldo del brazalete.
- Es la estructura idónea para mantener la trazabilidad de operaciones reversibles bajo la filosofía de "Último en Registrarse, Primero en Deshacerse".

6. Árbol Binario de Búsqueda Estándar (Módulo de Infraestructura)

- **Dónde se implementa:** En el catálogo del mapa temático del parque.
- **Cómo se usa:** Organiza las zonas y sub-zonas de manera alfabética o por códigos de ubicación. Al ser un árbol binario normal, servirá para que el estudiante note la diferencia de eficiencia frente al AVL cuando los datos se insertan de forma pre-ordenada (provocando que el árbol se degenera en una lista si no está balanceado).
- Permite realizar recorridos (*Inorden* para ver las zonas alfabéticamente, *Postorden* para dismantelar o cerrar zonas completas liberando memoria).